

Vibe Coding workflow分享



基于 Claude Code ，使用 superpowers + karpathy 两个skill作为示例

一、两个 Skill 介绍

<https://github.com/obra/superpowers> 插件

提供一整套 Vibe Coding workflow，核心 skill 包括：

- **brainstorming** — 逐个追问澄清需求，提出方案，产出 spec 设计文档
- **writing-plans** — 基于 spec 生成结构化执行计划
- **dispatching-parallel-agents / subagent-driven-development** — 多智能体并行执行代码改动

完整链路：想法 → spec → 执行计划 → 代码。

[andrej-karpathy-skills](#)插件

基于 Andrej Karpathy 总结的 LLM 编码常见错误，用四条规则审查计划：

规则	核心要求
Think Before Coding	显式陈述假设，有歧义就停下来问，不替用户做选择
Simplicity First	只写解决当前问题的最小代码，不加功能、不加抽象、不加防御性处理
Surgical Changes	只改必须改的，不动周边代码，风格与现有代码保持一致
Goal-Driven Execution	每步定义可验证的成功标准，形成「步骤 → 验证」的执行循环

二、为什么在 superpowers 流程中插入 karpathy?

superpowers 解决「从想法到代码」的完整链路，但生成执行计划时 LLM 容易过度设计或夹带无关改动。karpathy 作为 spec 和执行计划之间的审查关卡，用四条规则过滤这些问题：

- 没过 brainstorming → 容易做错东西
- 没过 karpathy 审查 → 容易做坏现有代码
- 两者配合 → 产出既对又稳

三、安装

claude code安装：

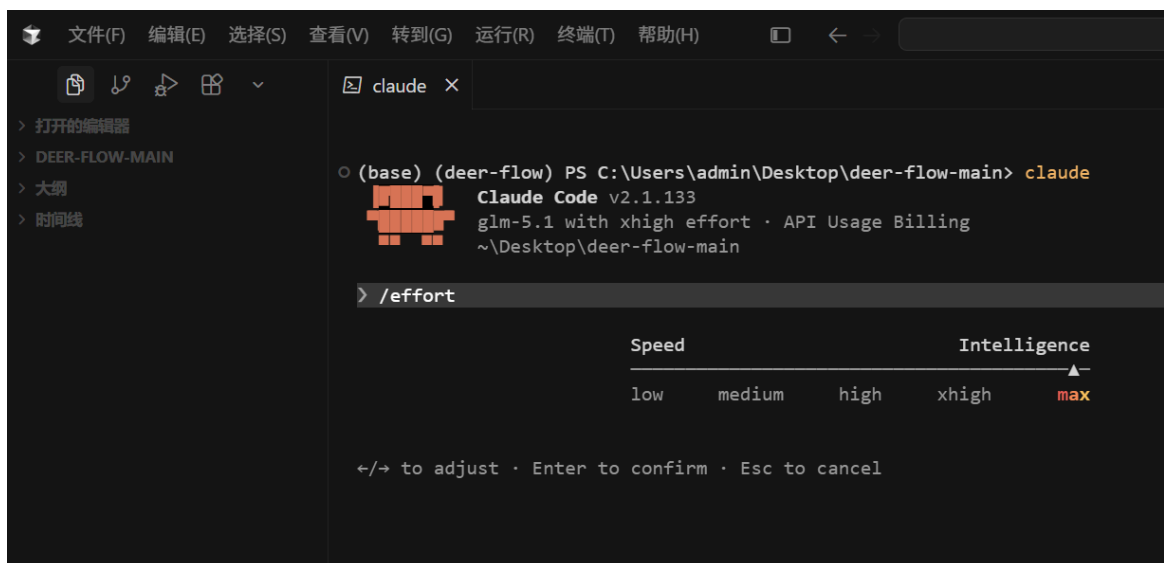
建议使用 cli 版本（对 skill 和 subagent 的支持性更好），安装教程：<https://claude-zh.cn/guide/getting-started>

由于 cli 版本的 Claude Code 没有图形界面，所以建议在 VSCode 的终端中使用，并安装 `Claude Code for VS Code` 插件方便对照代码查看 AI 的改动。

在终端中执行 `claude` 启动 Claude Code



为实现最佳性能，使用 `/effort` 命令调整模型思考模式为 `max`





安装 Claude Code for VS Code 插件的作用——新标签页展示代码改动

```
claude x  * [Claude Code] hello_world.py (92c915)  𐀀
o  Claude Code v2.1.133
   glm-5.1 with xhigh effort · API Usage Billing
   ~\Desktop\deer-flow-main

> 写一个hell word

• 我来写一个 Hello World 程序。先看看项目的情况。

  Listed 3 directories (ctrl+o to expand)

• 这是一个 Python 后端项目。我直接创建一个 hello_world.py。

• Write(deer-flow\backend\hello_world.py)

Opened changes in Cursor  𐀀

Save file to continue...

Do you want to make this edit to hello_world.py?
> 1. Yes
   2. Yes, allow all edits during this session (shift+tab)
   3. No

Esc to cancel · Tab to amend
```

```
claude  * [Claude Code] hello_world.py (92c915)  x
C: > Users > admin > Desktop > deer-flow-main > deer-flow > backend >  hello_world.py
1+ print("Hello, World!")
2+
1
```

skill 安装（安装命令均在 Claude Code 对话中执行）：

安装 superpowers：

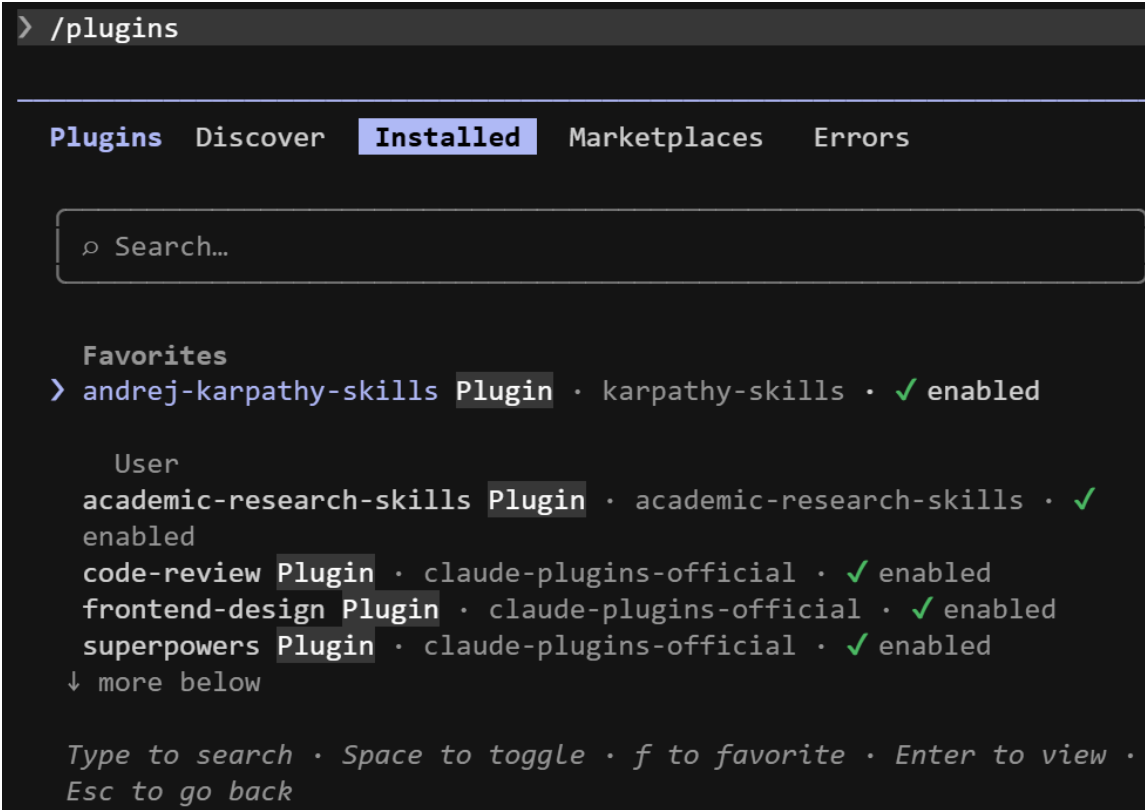
```
/plugin install superpowers@claude-plugins-official
```

安装 andrej-karpathy-skills：

```
/plugin marketplace add forrestchang/andrej-karpathy-skills  
/plugin install andrej-karpathy-skills@karpathy-skills
```

安装后用 `/plugin` 确认列表中出现 `rsuperpowers` 和 `karpathy`。

```
> /plugins
```



The screenshot shows the Claude Code interface with the `/plugins` command executed. The `Installed` tab is selected, displaying a list of installed plugins. The list includes `andrej-karpathy-skills Plugin`, `academic-research-skills Plugin`, `code-review Plugin`, `frontend-design Plugin`, and `superpowers Plugin`, all marked as `enabled`. A search bar is visible at the top, and a 'more below' link is at the bottom of the list.

```
Plugins Discover Installed Marketplaces Errors
```

Search...

Favorites

- > andrej-karpathy-skills Plugin · karpathy-skills · ✓ enabled
- User
- academic-research-skills Plugin · academic-research-skills · ✓ enabled
- code-review Plugin · claude-plugins-official · ✓ enabled
- frontend-design Plugin · claude-plugins-official · ✓ enabled
- superpowers Plugin · claude-plugins-official · ✓ enabled

↓ more below

Type to search · Space to toggle · f to favorite · Enter to view · Esc to go back

四、具体流程

第一步：brainstorming 生成 spec 文档

```
/brainstorming 在这里输入你的需求
```



小提示：这里按 TAB，而不是 Enter，Enter 会导致没有输入需求就直接执行头脑风暴

```
> /brain

/brainstorming      (superpowers) You MUST use this before
                    any creative work - creating features, ...
/branch             Create a branch of the current
                    conversation at this point
/finishing-a-development-... (superpowers) Use when implementation is
                    complete, all tests pass, and you need ...
/security-review     Complete a security review of the
                    pending changes on the current branch
```

```
> /superpowers:brainstorming 这里输入你的需求

glm-5.1 | Effort: max | Ctx: null%
```

调用示例：

```
/brainstorming 我想给项目加一个用户登录功能
```

Claude Code 会通过苏格拉底式提问，把需求细节挖干净：登录方式？注册流程？token 存哪里？... 回答完后提出 2-3 个方案，确认后写入 spec 文档：

```
docs/superpowers/specs/2026-XX-XX-XXXXXXX-design.md
```

产出物： 经过双方确认的设计文档。

第二步：karpathy 审查 spec 文档

spec 完成后、生成执行计划之前，调用 karpathy 审查，示例如下：

```
调用 karpathy skill 审查这份 spec，检查是否违反四条规则
```

karpathy 会检查：

- 是否有未声明的假设或歧义（Think Before Coding）
- 方案是否过度复杂，能否更简单（Simplicity First）
- 改动范围是否最小化（Surgical Changes）

- 每步是否有明确的成功标准（Goal-Driven Execution）

审查通过后再进入下一步。如果有问题，回到 spec 修正后重新审查。

第三步：生成执行计划

审查通过后，superpowers 的 `writing-plans` skill 基于 spec 生成详细执行计划 `docs/superpowers/specs/2026-XX-XX-XXXXXX-implementation plan.md`，每步包含具体改动内容和验证条件。

第四步：多智能体并行执行

计划中往往有多个独立任务（前端组件、后端接口、测试用例），使用多智能体并行完成。每个 agent 独立领取任务、独立执行、独立验证。完成后审查各 agent 产出，合并或调整。

产出物： 经过验证的代码改动。

完整流程图

```
模糊想法
↓
[brainstorming] 逐个提问 → 方案选择 → spec 文件
↓
[karpathy 审查] 四条规则检查 → 通过 / 打回修正
↓
[writing-plans] 生成执行计划（每步有验证标准）
↓
[多智能体执行] 并行编码 → 独立验证 → 审查合并
↓
可交付的代码
```

五、附录——Claude Code CLI 常用技巧

命令	作用
<code>/context</code>	查看当前上下文占用（token 用量、已加载的 skill/agent）。上下文健康度较低时，如果不想放弃当前上下文，可以在完成一个小的阶段性目标之后使用 <code>/compact</code> 命令主动压缩上下文。
<code>/clear</code>	清空当前对话，从头开始
<code>/compact</code>	压缩对话历史，释放上下文空间（长对话时有用）
<code>/resume</code>	恢复上一次中断的会话
<code>/effort</code>	调整模型思考深度（建议设为 <code>max</code> ）

命令	作用
<code>/skills</code>	查看当前可用的 skill 列表
<code>/plugin</code>	安装或卸载插件